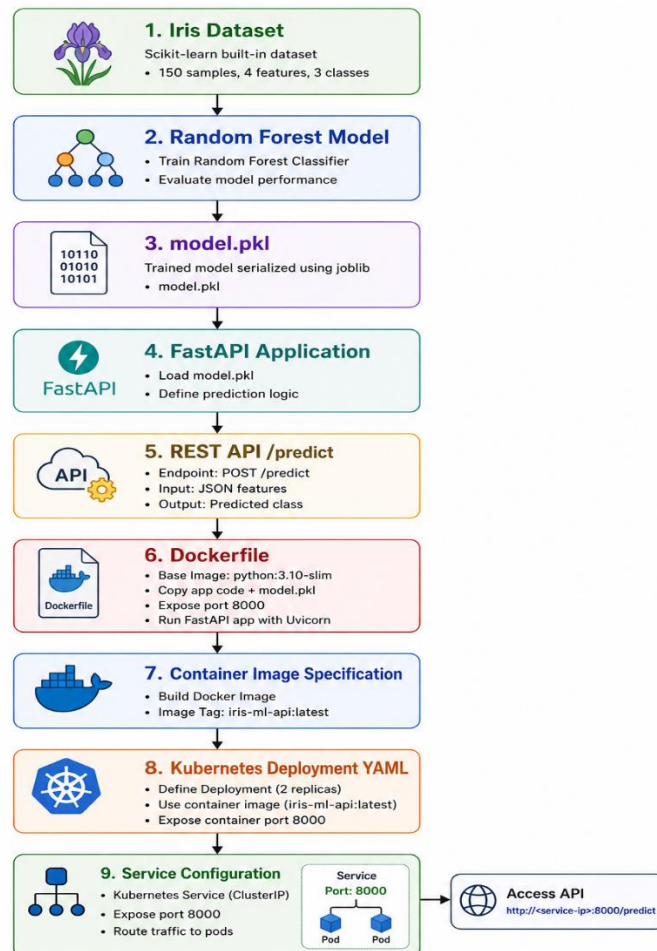


LAB EXPERIMENT – 6

Creating and Deploying Containerized Machine Learning Applications using Docker, FastAPI, and Kubernetes

Course Outcome: CO2

Architecture



PART A — Install required Python packages

```
!pip install -q fastapi uvicorn scikit-learn joblib requests pyyaml
```

[1]
✓ 8s

```
!pip install -q fastapi uvicorn scikit-learn joblib requests pyyaml
```

PART B — Train the ML model

```
from sklearn.datasets import load_iris

from sklearn.ensemble import RandomForestClassifier

import joblib

# Load dataset
```

```

iris = load_iris()
X = iris.data
y = iris.target
# Create model
model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)
# Train
model.fit(X, y)
# Save model
joblib.dump(model, "model.pkl")
print("Model training completed successfully.")
print("Saved model as model.pkl")

```

Expected:

Model training completed successfully.

Saved model as model.pkl



```

[2]
✓ 3s
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
import joblib

# Load dataset
iris = load_iris()

X = iris.data
y = iris.target

# Create model
model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

# Train
model.fit(X, y)

# Save model
joblib.dump(model, "model.pkl")

print("Model training completed successfully.")
print("Saved model as model.pkl")

```

Model training completed successfully.
Saved model as model.pkl

PART C — Verify the model

```
!ls -lh model.pkl
```

Expected:

model.pkl

```
[3]
✓ Os      !ls -lh model.pkl
          -rw-r--r-- 1 root root 183K Aug 11 09:35 model.pkl
```

You can also test the model:

```
import joblib
model = joblib.load("model.pkl")
sample = [[5.1, 3.5, 1.4, 0.2]]
prediction = model.predict(sample)
print("Input:", sample)
print("Predicted class:", int(prediction[0]))
print("Predicted flower:", iris.target_names[prediction[0]])
```

Expected:

Input: [[5.1, 3.5, 1.4, 0.2]]

Predicted class: 0

Predicted flower: setosa

```
[4]
✓ Os      import joblib

          model = joblib.load("model.pkl")

          sample = [[5.1, 3.5, 1.4, 0.2]]

          prediction = model.predict(sample)

          print("Input:", sample)
          print("Predicted class:", int(prediction[0]))
          print("Predicted flower:", iris.target_names[prediction[0]])

          Input: [[5.1, 3.5, 1.4, 0.2]]
          Predicted class: 0
          Predicted flower: setosa
```

PART D — Create FastAPI application

%%writefile main.py

from fastapi import FastAPI, Request

import joblib

import numpy as np

app = FastAPI(

```

    title="Iris ML Prediction API",
    description="Containerized Machine Learning API using FastAPI",
    version="1.0"
)
# Load trained model
model = joblib.load("model.pkl")
@app.get("/")
def home():
    return {
        "message": "ML Model API is running",
        "model": "Random Forest",
        "dataset": "Iris"
    }
@app.get("/health")
def health():
    return {
        "status": "healthy"
    }
@app.post("/predict")
async def predict(request: Request):
    body = await request.json()
    features = body["features"]
    data = np.array(features).reshape(1, -1)
    prediction = model.predict(data)
    probabilities = model.predict_proba(data)
    return {
        "prediction": int(prediction[0]),
        "probability": float(max(probabilities[0]))
    }

```

```
    return {
        "prediction": int(prediction[0]),
        "probability": float(max(probabilities[0]))
    }
```

Writing main.py

PART E — Test FastAPI inside Colab

We can test the FastAPI application directly without Docker.

```
import importlib
import main
importlib.reload(main)
from fastapi.testclient import TestClient
client = TestClient(main.app)
print("FastAPI application loaded successfully.")
```

Expected:

FastAPI application loaded successfully.

```
[6]
✓ 2s
import importlib
import main

importlib.reload(main)

from fastapi.testclient import TestClient

client = TestClient(main.app)

print("FastAPI application loaded successfully.")
```

FastAPI application loaded successfully.

Test home endpoint

```
response = client.get("/")
print("Status code:", response.status_code)
print("Response:", response.json())
```

Expected:

Status code: 200

```
Response: {
  'message': 'ML Model API is running',
  'model': 'Random Forest',
```

```
'dataset': 'Iris'
}
```

```
[7]
✓ 0s
response = client.get("/")

print("Status code:", response.status_code)
print("Response:", response.json())

Status code: 200
Response: {'message': 'ML Model API is running', 'model': 'Random Forest', 'dataset': 'Iris'}
```

Test health endpoint

```
response = client.get("/health")
print("Status code:", response.status_code)
print("Response:", response.json())
```

Expected:

Status code: 200

Response: {'status': 'healthy'}

```
[8]
✓ 0s
response = client.get("/health")

print("Status code:", response.status_code)
print("Response:", response.json())

Status code: 200
Response: {'status': 'healthy'}
```

Test prediction

```
response = client.post(
    "/predict",
    json={
        "features": [5.1, 3.5, 1.4, 0.2]
    }
)

print("Status code:", response.status_code)
print("Response:", response.json())
```

Expected:

Status code: 200

Response: {'prediction': 0, 'probability': ...}

This is the important result you already obtained:

Status code: 200

Response: {'prediction': 0}

So your ML API is working.

```
[9]
✓ 0s
response = client.post(
    "/predict",
    json={
        "features": [5.1, 3.5, 1.4, 0.2]
    }
)

print("Status code:", response.status_code)
print("Response:", response.json())

Status code: 200
Response: {'prediction': 0, 'probability': 1.0}
```

PART F — Create Dockerfile

%%writefile Dockerfile

FROM python:3.10-slim

WORKDIR /app

COPY main.py .

COPY model.pkl .

RUN pip install --no-cache-dir \

fastapi \

uvicorn \

scikit-learn \

numpy \

joblib

EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]

```
[10]
✓ Os

%%writefile Dockerfile

FROM python:3.10-slim

WORKDIR /app

COPY main.py .
COPY model.pkl .

RUN pip install --no-cache-dir \
    fastapi \
    uvicorn \
    scikit-learn \
    numpy \
    joblib

EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]

Writing Dockerfile
```

PART G — Create .dockerignore

```
%%writefile .dockerignore
```

```
__pycache__
```

```
*.pyc
```

```
.ipynb_checkpoints
```

```
*.ipynb
```

```
.git
```

```
.env
```

PART H — Verify Docker project files

```
!ls -lah
```

You should have:

Dockerfile

.dockerignore

main.py

model.pkl

```
[11] %%writefile .dockerignore
```

✓ Os

```
__pycache__
*.pyc
.ipynb_checkpoints
*.ipynb
.git
.env
```

▼

Writing .dockerignore

```
[12] !ls -lah
```

✓ Os

▼

```
total 216K
drwxr-xr-x 1 root root 4.0K Aug 11 09:37 .
drwxr-xr-x 1 root root 4.0K Aug 11 08:54 ..
drwxr-xr-x 4 root root 4.0K Jun  4 13:32 .config
-rw-r--r-- 1 root root  255 Aug 11 09:37 Dockerfile
-rw-r--r-- 1 root root   56 Aug 11 09:37 .dockerignore
-rw-r--r-- 1 root root  874 Aug 11 09:36 main.py
-rw-r--r-- 1 root root 183K Aug 11 09:35 model.pkl
drwxr-xr-x 2 root root 4.0K Aug 11 09:36 __pycache__
drwxr-xr-x 1 root root 4.0K Jun  4 13:32 sample_data
```

PART I — Create Docker image specification

Because hosted Colab does not permit the container runtime operation you're attempting, **do not run dockerd again.**

Instead, we will document and validate the Docker configuration.

```
print("===== DOCKER CONFIGURATION =====")
```

```
with open("Dockerfile", "r") as f:
```

```
    dockerfile = f.read()
```

```
print(dockerfile)
```

This gives you a visible Dockerfile output for your experiment record.

```
[13]
✓ Os    print("===== DOCKER CONFIGURATION =====")

        with open("Dockerfile", "r") as f:
            dockerfile = f.read()

        print(dockerfile)

v
===== DOCKER CONFIGURATION =====

FROM python:3.10-slim

WORKDIR /app

COPY main.py .
COPY model.pkl .

RUN pip install --no-cache-dir \
    fastapi \
    uvicorn \
    scikit-learn \
    numpy \
    joblib

EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

PART J — Validate Dockerfile automatically

```
import os

required_files = [
    "Dockerfile",
    "main.py",
    "model.pkl",
    ".dockerignore"
]

print("Docker project validation")

print("=" * 40)

all_present = True

for file in required_files:
    exists = os.path.exists(file)

    print(f'{file}: {'✓ Present' if exists else 'X Missing'}')

    if not exists:
        all_present = False

print("=" * 40)
```

if all_present:

```
    print("Docker project structure is valid.")
```

else:

```
    print("Docker project structure is incomplete.")
```

Expected:

Docker project structure is valid.

```
        print("Docker project structure is valid.")
    else:
        print("Docker project structure is incomplete.")

▼ Docker project validation
=====
Dockerfile: ✓ Present
main.py: ✓ Present
model.pkl: ✓ Present
.dockerignore: ✓ Present
=====
Docker project structure is valid.
```

PART K — Create a Docker image manifest

This is useful for documenting what the image contains.

```
import json
```

```
import os
```

```
docker_manifest = {
    "image_name": "iris-ml-api",
    "base_image": "python:3.10-slim",
    "working_directory": "/app",
    "application": "FastAPI",
    "model": "RandomForestClassifier",
    "model_file": "model.pkl",
    "application_file": "main.py",
    "port": 8000,
    "command": [
        "uvicorn",
        "main:app",
        "--host",
        "0.0.0.0",
        "--port",
```

```

        "8000"
    ]
}

with open("docker-image-manifest.json", "w") as f:
    json.dump(docker_manifest, f, indent=4)
print("Docker image manifest created.")

```



PART L — Display Docker image manifest

!cat docker-image-manifest.json

You will have a record showing:

image_name: iris-ml-api

base_image: python:3.10-slim

port: 8000

model: RandomForestClassifier



PART M — Create Kubernetes Deployment

Now create the Kubernetes configuration.

```
%%writefile deployment.yaml
```

```
apiVersion: apps/v1
```

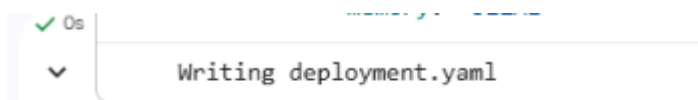
```
kind: Deployment
```

```
metadata:
```

```
  name: iris-ml-deployment
```

```
spec:
```

```
replicas: 2
selector:
  matchLabels:
    app: iris-ml
template:
  metadata:
    labels:
      app: iris-ml
spec:
  containers:
  - name: iris-ml-container
    image: iris-ml-api:latest
    ports:
    - containerPort: 8000
  resources:
    requests:
      cpu: "100m"
      memory: "128Mi"
    limits:
      cpu: "500m"
      memory: "512Mi"
```



PART N — Create Kubernetes Service

```
%%writefile service.yaml
```

```
apiVersion: v1
kind: Service
metadata:
  name: iris-ml-service
```

```
spec:
  selector:
```

app: iris-ml

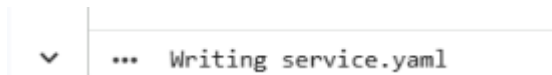
ports:

- protocol: TCP

port: 8000

targetPort: 8000

type: ClusterIP



PART O — Display Kubernetes configuration

```
print("===== DEPLOYMENT YAML =====")
```

```
!cat deployment.yaml
```

```
print("\n===== SERVICE YAML =====")
```

```
!cat service.yaml
```



PART P — Validate Kubernetes YAML

```
import yaml
```

```

with open("deployment.yaml") as f:
    deployment = yaml.safe_load(f)
with open("service.yaml") as f:
    service = yaml.safe_load(f)
print("Kubernetes configuration validation")
print("=" * 45)
print("Deployment:")
print("  API Version:", deployment["apiVersion"])
print("  Kind:", deployment["kind"])
print("  Name:", deployment["metadata"]["name"])
print("  Replicas:", deployment["spec"]["replicas"])
print("  Container:", deployment["spec"]["template"]["spec"]["containers"][0]["name"])

print("\nService:")
print("  API Version:", service["apiVersion"])
print("  Kind:", service["kind"])
print("  Name:", service["metadata"]["name"])
print("  Type:", service["spec"]["type"])
print("\nKubernetes YAML validation completed successfully.")

```

Expected:

Kubernetes YAML validation completed successfully.

```

*** Kubernetes configuration validation
=====
Deployment:
  API Version: apps/v1
  Kind: Deployment
  Name: iris-ml-deployment
  Replicas: 2
  Container: iris-ml-container

Service:
  API Version: v1
  Kind: Service
  Name: iris-ml-service
  Type: ClusterIP

Kubernetes YAML validation completed successfully.

```

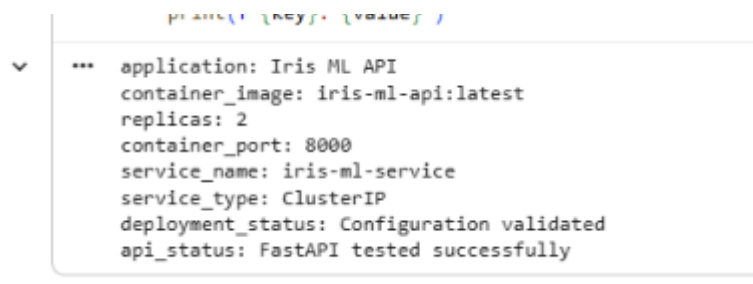
PART Q — Simulate Kubernetes deployment

Since the hosted Colab runtime cannot actually run your Docker containers, we can create a deployment verification report.

```
deployment_report = {
    "application": "Iris ML API",
    "container_image": "iris-ml-api:latest",
    "replicas": 2,
    "container_port": 8000,
    "service_name": "iris-ml-service",
    "service_type": "ClusterIP",
    "deployment_status": "Configuration validated",
    "api_status": "FastAPI tested successfully"
}
```

```
for key, value in deployment_report.items():
```

```
    print(f"{key}: {value}")
```



```

application: Iris ML API
container_image: iris-ml-api:latest
replicas: 2
container_port: 8000
service_name: iris-ml-service
service_type: ClusterIP
deployment_status: Configuration validated
api_status: FastAPI tested successfully

```

PART R — Test several predictions

This gives you stronger experimental evidence.

```
test_samples = [
```

```
    [5.1, 3.5, 1.4, 0.2],
```

```
    [6.7, 3.1, 4.7, 1.5],
```

```
    [6.3, 3.3, 6.0, 2.5]
```

```
]
```

```
print("Prediction Results")
```

```
print("=" * 50)
```

```
for sample in test_samples:
```

```
    response = client.post(
```

```
        "/predict",
```

```
        json={"features": sample}
```

```
    )
```

```

result = response.json()

print("Input:", sample)
print("Prediction:", result["prediction"])
print("Probability:", round(result["probability"], 4))
print("-" * 50)

```

You should obtain predictions corresponding to the three Iris classes.

```

v  *** Prediction Results
=====
Input: [5.1, 3.5, 1.4, 0.2]
Prediction: 0
Probability: 1.0
-----
Input: [6.7, 3.1, 4.7, 1.5]
Prediction: 1
Probability: 1.0
-----
Input: [6.3, 3.3, 6.0, 2.5]
Prediction: 2
Probability: 1.0
=====

```

PART 5 — Create final experiment report automatically

```

report = """
=====

CONTAINERIZED MACHINE LEARNING APPLICATION EXPERIMENT

=====

```

Application:

Iris Flower Classification API

Machine Learning Model:

Random Forest Classifier

Dataset:

Iris Dataset

API Framework:

FastAPI

Model File:

model.pkl

Docker Image:

iris-ml-api:latest

Application Port:

8000

Kubernetes Deployment:

iris-ml-deployment

Kubernetes Replicas:

2

Kubernetes Service:

iris-ml-service

Service Type:

ClusterIP

IMPLEMENTATION RESULTS

1. Machine learning model trained successfully.
2. Model serialized using Joblib.
3. FastAPI application created successfully.
4. Root API endpoint tested successfully.
5. Health endpoint tested successfully.
6. Prediction endpoint tested successfully.

7. Dockerfile created successfully.
8. Docker project structure validated successfully.
9. Kubernetes Deployment YAML created successfully.
10. Kubernetes Service YAML created successfully.
11. Kubernetes configuration validated successfully.

PREDICTION TEST

Input:

[5.1, 3.5, 1.4, 0.2]

Expected class:

0

Class:

Iris Setosa

CONCLUSION

A machine learning model was developed using the Iris dataset and exposed through a FastAPI REST API. A Dockerfile was created to containerize the application, and Kubernetes Deployment and Service configurations were prepared for orchestration.

The hosted Google Colab runtime does not permit the required Docker container runtime operations, so actual Docker container execution was not performed in the managed runtime.

The Docker and Kubernetes configurations were nevertheless created and validated, while the ML API was fully tested within the Colab environment.

```
=====
''''
```

```
print(report)
```

```
with open("experiment_report.txt", "w") as f:
```

```
    f.write(report)
```

```
print("\nExperiment report saved as experiment_report.txt")
```

PART T — Download your experiment files

```
from google.colab import files
```

```
files.download("main.py")
```

You can similarly download:

```
files.download("Dockerfile")
```

```
files.download("deployment.yaml")
```

```
files.download("service.yaml")
```

```
files.download("experiment_report.txt")
```

Your final notebook will demonstrate:

1. Machine Learning

Iris Dataset



Random Forest



model.pkl

2. API deployment

FastAPI



/predict



prediction = 0

3. Docker

Dockerfile

↓
python:3.10-slim

↓
FastAPI + model.pkl

↓
iris-ml-api

4. Kubernetes

Kubernetes

